DATA 427 Group Project: Spark ML on the KDD Dataset

Jason Pollock, Fabian Day, Bex Campbell-Redl

1 Implement two programs that apply Spark's Decision Tree algorithm and Logistic Regression algorithm to the provided KDD dataset. Run these programs 10 times on the school's Hadoop cluster, each using a different seed to split the dataset into a training and a test set.

1.1 Describe the two programs using pseudo-code

1.1.1 Logistic Regression:

Algorithm 1 Spark Logistic Regression

Parse the input/output directories and number of runs
Initialise the Spark session and load the data from the provided input directory

Setup the pipeline as follows:
Index required columns
One-hot encode required columns
Assemble into feature columns
Append parameterised logistic regression model as the final stage.
for Number of runs **do**
    Set seed
    Split the data into train/test
    Fit the LR model on the train data
    Evaluate the models performance on the training and testing split, and record results
end for
Display results

1.1.2 Decision Tree

Algorithm 2 Spark Decision Tree

Parse the input/output directories and number of runs
Initialise the Spark session and load the data from the provided input directory
Generate schema
Setup the pipeline as follows:
Index required columns
Assemble into feature columns
Append parameterised DecisionTree model as the final stage.
Run the pipeline:
Split the test/training data into num_runs instances
Foreach training data, train a model
Evaluate the results:
Output the DecisionTree structure for the first tree
Foreach (model, training data) pair, generate predictions
Foreach (model, testing data) pair, generate predictions
Foreach training prediction, generate an accuracy row
Foreach test prediction, generate an accuracy row
Produce a CSV with the aggregate data by test/train
Produce a CSV with the per-run data
Display results

1.2 Describe how to install and run the two programs step by step in a Readme.txt file.

See project folder.

1.3 Report the training and test results including the max, min, average accuracy and the standard deviation obtained from the 10 runs, and the running time of each program.

1.3.1 Decision Tree:

The Decision Tree produced consistently high test accuracy (mean 94.54%, SD0.20) across the ten runs, the spread of training to test gaps was small enough that sometimes the test score was higher than the training score. This could be because the depth cap (spark default depth=5) prevented overfitting, with the remaining variation coming mostly from the random split rather than from the model. The top of the learned tree consistently splits on features 4, 35 and 39 which indicates that a few features contain the most relevant information.

Table 1: Summary of decision tree accuracy over 10 runs.

Metric	Average	Std. dev.	Min	Max
Training accuracy (%)	94.62	0.15	94.39	94.94
Test accuracy (%)	94.57	0.21	94.37	95.06

1.3.2 Logistic Regression:

Logistic Regression performed slightly worse (mean test accuracy 94.15%, SD0.17) and had a slightly bigger test train gap of 0.2. The categorical columns were one hot encoded so the gap

is unlikely to be a preprocessing artefact, it is potentially a result of Logistic Regression's limited ability to capture non-linear interactions between features that the Decision Tree can capture with successive splits. The model had stable performance across seeds which suggests that the gap between the training and test performance is due to the model itself, not a specific train/test partition.

Table 2: Summary of logistic regression accuracy over 10 runs.

Metric	Average	Std. dev.	Min	Max
Training accuracy (%)	94.35	0.11	94.21	94.50
Test accuracy (%)	94.15	0.17	93.93	94.37

1.3.3 Comparison

In this case the Decision Tree is more accurate and approximately twice as fast as Logistic Regression (Decision Tree test accuracy ranged from 94.37% - 95.06% with a run time of 65.4s and Logistic Regression test accuracy ranged from 93.93% - 94.37% with a run time of 129.3s).The test accuracy gap between DT and LR could be explained by non linear feature interactions within the data that the Decision Tree can capture in tree construction that Logistic Regression cannot. Or as a result of the lack of tuning of the regularisation parameter in the Logistic Regression. The difference in runtime is reflective of how the two algorithms are executed in spark.The decision tree can be optimised in its execution more than the logistic regression model which could potentially be the reason it ran faster. The Decision Tree model also has less features to process than the Logistic Regression model which was one hot encoded and could therefore have contributed to slower completion time. The decision tree output offers more interpretability and better handles mixed feature types.

We evaluated only a single hyperparameter configuration per model (Default depth for DT, Default settings for LR, 70/30 train-test split and 10 seeds), it is possible that further tuning of either model could shift the comparison in accuracy however the difference in time is a product of how the models perform in spark and is unlikely to change significantly.

Table 3: Summary of model test accuracy and total running time over 10 runs.

Program	Average acc. (%)	Std. dev.	Min acc. (%)	Max acc. (%)	Total time (s)
Decision Tree	94.57	0.21	94.37	95.06	65.37
Spark Logistic Regression	94.15	0.17	93.93	94.37	129.29

1.4 Appendix:

Table 4: Decision tree results for each seeded run.

Run	Seed	Train count	Test count	Train acc. (%)	Test acc. (%)
1	221	33372	14364	94.49	94.40
2	222	33414	14322	94.71	94.40
3	223	33434	14302	94.67	94.39
4	224	33373	14363	94.61	94.37
5	225	33484	14252	94.53	94.56
6	226	33351	14385	94.59	94.68
7	227	33502	14234	94.66	94.65
8	228	33167	14569	94.39	94.63
9	229	33494	14242	94.94	95.06
10	230	33482	14254	94.59	94.61

Table 5: Logistic regression results for each seeded run.

Run	Seed	Train count	Test count	Train acc. (%)	Test acc. (%)	Time (s)
1	1	33402	14334	94.35	94.06	10.09
2	2	33429	14307	94.24	94.07	7.93
3	3	33571	14165	94.36	94.35	7.09
4	4	33434	14302	94.50	94.37	7.04
5	5	33314	14422	94.47	94.03	6.63
6	6	33570	14166	94.30	94.16	6.61
7	7	33431	14305	94.30	93.93	6.69
8	8	33535	14201	94.21	94.37	6.47
9	9	33489	14247	94.27	94.20	6.40
10	10	33357	14379	94.50	93.95	6.37

1.4.1 Example of trained decision tree structure from a run

DecisionTreeClassificationModel: depth=5, numNodes=51, numClasses=2, numFeatures=41
if feature 4 $\leq$ 30.5 **then**
  if feature 35 $\leq$ 0.005 **then**
    if feature 5 $\leq$ 130.5 **then**
      if feature 1 $\leq$ 0.5 **then**
        Predict: 0.0
      else
        if feature 34 $\leq$ 0.015 **then**
          Predict: 1.0
        else
          Predict: 0.0
        end if
      end if
    else
      if feature 2 $\leq$ 15.5 **then**
        Predict: 1.0
      else
        if feature 32 $\leq$ 132.5 **then**

```
                Predict: 0.0
            else
                Predict: 1.0
            end if
        end if
    end if
else
    if feature 32 ≤ 168.5 then
        Predict: 0.0
    else
        if feature 4 ≤ 0.5 then
            if feature 2 ≤ 52.5 then
                Predict: 1.0
            else
                Predict: 0.0
            end if
        else
            if feature 36 ≤ 0.015 then
                Predict: 1.0
            else
                Predict: 0.0
            end if
        end if
    end if
end if
else
    if feature 39 ≤ 0.005 then
        if feature 1 ≤ 1.5 then
            if feature 0 ≤ 29.5 then
                Predict: 1.0
            else
                if feature 2 ≤ 17.5 then
                    Predict: 0.0
                else
                    Predict: 1.0
                end if
            end if
        else
            if feature 4 ≤ 195.5 then
                if feature 30 ≤ 0.685 then
                    Predict: 1.0
                else
                    Predict: 0.0
                end if
            else
                if feature 4 ≤ 1518.5 then
                    Predict: 0.0
                else
                    Predict: 1.0
                end if
            end if
```

```
        end if
    else
        if feature 36 ≤ 0.005 then
            if feature 4 ≤ 14714.5 then
                if feature 5 ≤ 608.5 then
                    Predict: 0.0
                else
                    Predict: 1.0
                end if
            else
                if feature 33 ≤ 0.615 then
                    Predict: 1.0
                else
                    Predict: 0.0
                end if
            end if
        else
            if feature 0 ≤ 2.5 then
                if feature 7 ≤ 0.5 then
                    Predict: 1.0
                else
                    Predict: 0.0
                end if
            else
                if feature 5 ≤ 130.5 then
                    Predict: 0.0
                else
                    Predict: 1.0
                end if
            end if
        end if
    end if
end if
```